Docker

Docker :-

→ Docker is an open-source centralised platform designed to create, deploy and run application.

→ Docker uses container on the host OS to run application it allows application to use the same linux kernel as a system on the host computer, rather than creating a whole virtual OS

→ We can install docker on any OS but Docker engine runs natively on linux distribution.

→ Docker written in 'go' language.

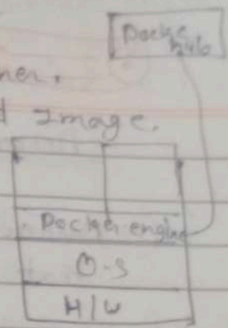
→ Docker is a tool that perform OS level virtualization, also known as containerization.

→ Before Docker, Many users face the problem that a particular code is running in the developer's system but not in a the user system.

→ Docker is first release in March 2013 it is developed by soloman Hykes and sebastian pahl.

→ Docker is a set of plateform as a service that uses OS Level virtualization where as vmware uses Hardware level virtualisation.

Image —Runing→ Container
Stop
stop
* Runing Image is called container.
* stop or pause container is called Image.



Advantages of Docker :-

(i) No pre-allocation of RAM.

(ii) CI Efficiency / contigous intrigation efficiency

Docker enables to build a container image and use that same across every step of the deployment process.

(iii) Low Cost

(iv) it is light in weight.
light weight - light weight means किसी Application को चलने के लिए बहुत कम resource का use करते हैं।

(v) it can run on physical H/w / virtual H/w or on cloud.

(vi) you can reuse
* we can not change image.

(vii) it took very less time to create container.

Disadvantages of Docker :-

(i) Docker is not a good solution for application that requires rich GUI.

(ii) Difficult to manage large amount of containers.

(iii) Docker does not provide cross-platform compatibility: means if an application is designed to run in a docker container on windows, then it can't run on linux or vice-versa.

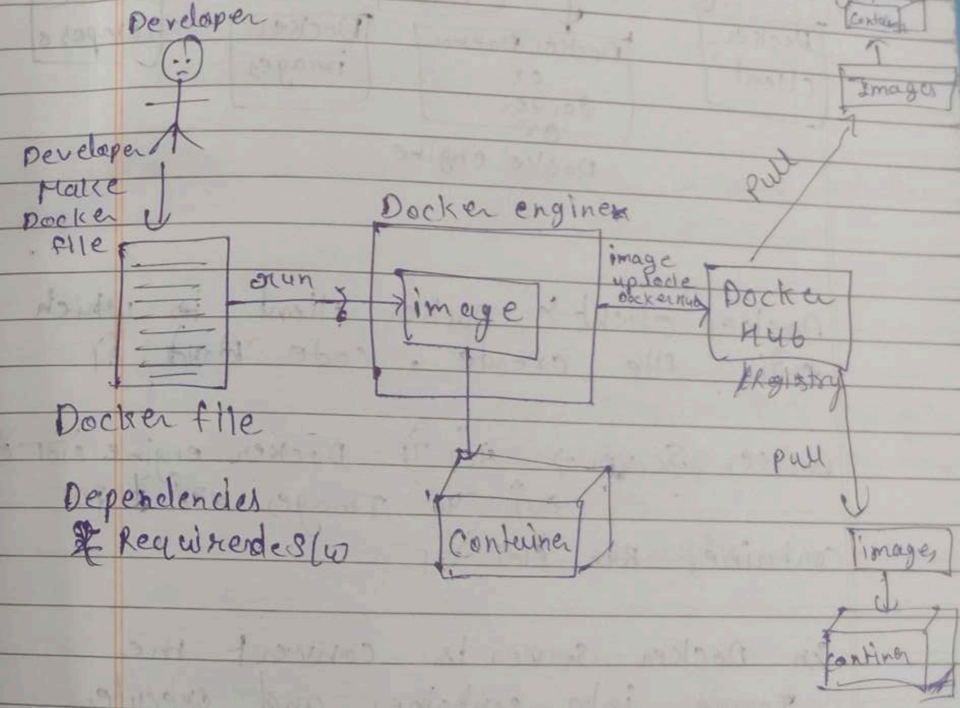
(iv) Docker is suitable when the development O.S and testing O.S are same. If the O.S is different, we should use VM.

(v) No solution for Data recovery & Backup.

* When the Docker file is Run in Docker engine then create image. और Docker engine उस image के एक copy को अपने पास रख लेता है।

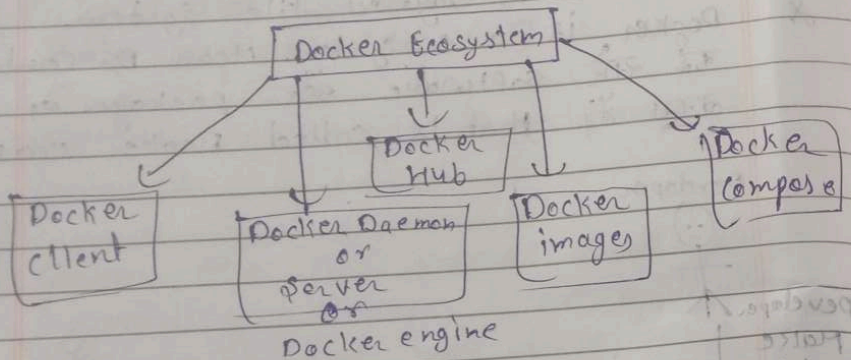
Architecture of Docker :-

* Container is layered file system.
* Docker is a ecosystem. Mean container कई सारे software और packages से मिल कर बनता है, that is called Docker ecosystem.



* docker is layer file system because it work step by step or layer form.

Docker Ecosystem :- set of software or package.



Docker client → Docker client is which ~~is~~ file create, code लिखते हैं।

Docker Server → यहाँ पे Docker engine होता है यहाँ पर Images बनती है तथा container Run होता है।

In Docker Server is convert the Image into container, and execute the command like pull, Run, etc.

Docker images → docker images is a file that contains the instructions for building a Docker container.

Docker Hub → Docker Hub is place जहाँ हम images को store करवाते हैं। it is kind of storage जहाँ पर ~~हम~~ Image store रहते हैं।

Docker images → Docker image से हम Docker container बनाते हैं। it is just like a Templates or AMI (Amzon Machine Image)

Docker Compose → बहुत सारे container को Run करवा सकते हैं।

Note

Docker Daemon →

- ~~Docker~~ Docker daemon runs on the Host O.S.
- It is responsible for running container to, Manages the docker servies
- Docker daemon can communicate with other Obemons.

Docker client -

Docker users can intract with docker ~~dom~~ daemon through a clients (CLI)

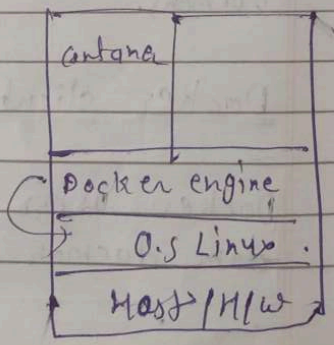
→ Docker client uses commands and Rest API to communicate with the docker daemon.

→ when a client runs any server command on the docker client terminal, the client terminal sends these docker command to the docker daemon.

→ it is possible for docker client to communicate with more than one daemon.

Docker Host →

Docker Host is used to provide an environment to execute and run application. it contain the docker ~~dam~~ daemon images, containers, networks and storage.



Docker Hub/ Registry :-

Docker Hub Manages and store the docker images.

There are two types of registries in the docker.

(i) Public registry - Public ~~registr~~ registry is also called as docker hub.

(ii) Private Registry - it is used to share images within the enterprise.

Docker Images -

Docker images are the read only binary templates used to ~~creat~~ create docker containers.

or

Single file with all dependencies and configuration required to run a program/ container.

Ways to create an Images.

* Take image from docker hub.
* Create image from docker file.
* create image from from existing docker container.

Docker Container :-

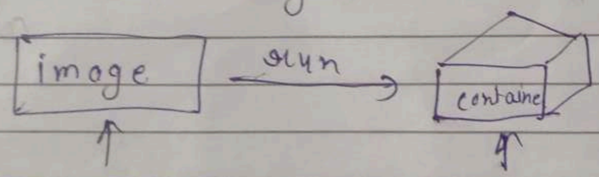
→ Container hold the entire packages that is needed to run the application.

or

In other words, we can say that the image is a template and the container is is a copy of that template.

→ Container like a virtual Machine.

→ images becomes container when they run on docker engine.



We can not Modifye. We can Modifye

yum install docker -y

Lecture - 26

→ Basic Commands in Docker :-

→ To see all images present in your local machine

Command → docker images [machine में जितनी images होंगी वह show करेगा docker hub की images नहीं दिखेगा].

→ To findout images in docker hub.

Syntax docker search images-name

Ex docker search ubuntu [Docker hub में जितनी ubuntu Jenkins की images होंगी वो दिखायेगा]

→ To download image from dockerhub to local Machine.

Syntax docker pull images-name

Ex docker pull Jenkins [Docker hub से image को Machine में लाता है]

Lecture-26

(4) To give name to Container (terminal)

Ex:- docker run -it --name Rajni ubuntu /bin/bash

(create + start) (intractive mode) (container mode)

(5) To check service is start or not

Command -

service docker status

(6) To start container

syntax

docker start container_name

Command: docker start Rajni

(7) To go inside Container { Container में अंदर जाने के लिए }

Command ⇒ docker attach Rajni { container name }

(8) To see all Containers

Command ⇒ docker ps -a

(9.) To see only running Containers.

→ docker ps
↓ (process status)

(10) To stop container

→ docker stop Rajni → (container name)

(11) To delete container

→ docker rm Rajni → (container name)

(12) Shows contain information

→ cat /bin/os release

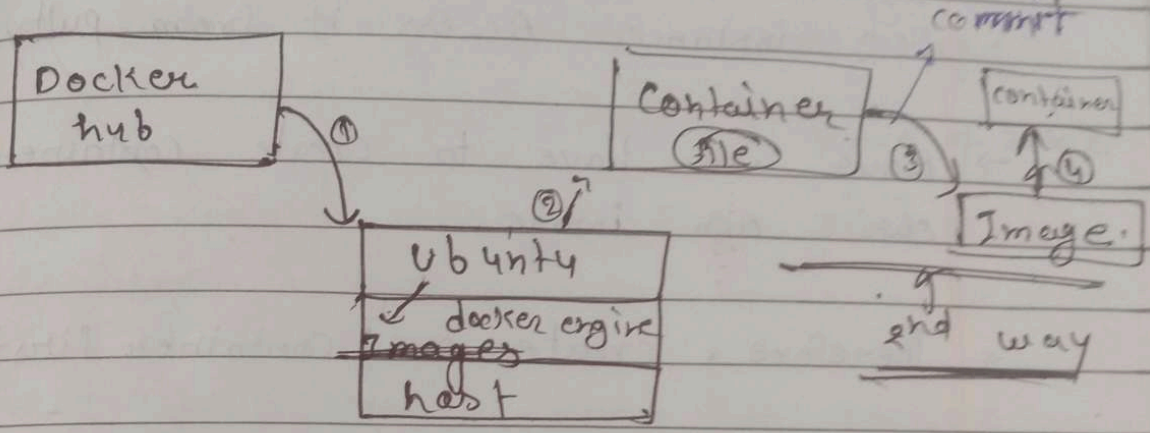
Lecture - 27

1. docker hub से Image बनाना
2. container से Image बनाना
3. docker file में build command से Image बनाना

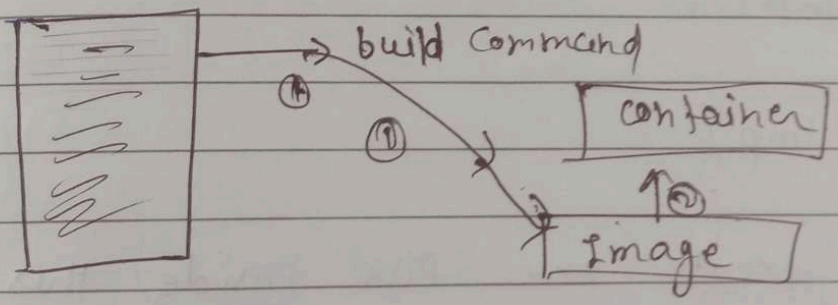
Way of docker Images creation

First way

1



3 way Docker file



Lecture-27

Docker file components & diff command

→ Login into AWS account and start your EC2 instance. Access it from putty.

→ Now we have to create container from our own image.

→ Therefore, create one container first

→ docker run -it --name bhupicontainer ubuntu /bin/bash

→ cd tmp/

Now create one file inside this tmp directory

→ touch myfile

Now if you want to see the difference between the base image & change on it then

→ docker diff ~~bh~~ bhupicontainer ~~update image~~

O/P →

C	/root
A	/root/bash-history
C	/tmp
A	/tmp/myfile

D → Deletion
C → change
A → Append/Addition

Commit means हम अपने contain से Image बना रहे हैं।

Container से Image बनाते हैं।

Now create image of the container

→ docker commit ~~newcontainer~~ bhupicontainer update_image

(↑ Container_name, ↑ Name of Image)

→ docker image (To check Images बने है कि नहीं)

Update_Image latest
ubuntu latest

Now create container from this image

→ docker run -it --name rajcontainer update_image /bin/bash

→ root@cd # ls
cd tmp/
tmp # ls

O/P → myfile {you will get all file back}

LAB 27

इसी तरह देना है case sensitive

Docker file

→ Dockerfile is basically a text file. it contains some set of Instruction.

→ Automation of Docker image creation.

Note → Dockerfile is name is case sensitive.

Docker Components

Note → Docker components is allways write capital letter.
Ex - FROM , RUN , COPY etc.

Components :-

① FROM → For base image This command must be on top of the dockerfile.

• docker file के पहली command FROM ही होती है

② RUN → To execute commands, it will create a layer in image.

③ MAINTAINER → Auther / lower / Description
Maintainer जो Image बना रहा है वही Maintener होता है।

Auther - जो Docker file बना रहा है
Description - it mean वो कौन से company से है Description में उसके detail रहती है।

→ Maintainer में Author के full information होती है।

④ COPY → Copy file from local system (docker VM) we need to provide source, destination.
(we can't download file Internet and any remote repo)

⑤ ADD → Similar to COPY but, it provides a feature to download file from internet, also we extract file at docker image side.

⑥ EXPOSE → To expose ports such as port 8080 for tomcat, port 80 for nginx etc.

⑦ WORKDIR → To set working directory for a container.
container बनने के बाद सीधे हम उसी directory में पहुँच जायेंगे। जो भी हम WORKDIR में देंगे।

(8) CMD → Execute commands but during Container creation.

(9) ENTRYPOINT → Similar to CMD, but has higher priority over CMD, first commands will be executed by ENTRYPOINT only.

(10) ENV → Environment Variables
Ex- Primeminister = Narendra Modi

(11) ARG →

LAB - 2

(i) Create a file named Dockerfile.

(ii) Add instructions in Docker file.
(iii) Build dockerfile to create image.
(iv) Run image to create container.

Ex step (1) → vi Dockerfile [[illegible] docker image [illegible]]

step (2) Instruction
{FROM ubuntu → {ubuntu base os [illegible]}
{RUN echo "Technical guftgu" >/tmp/testfile

To create image out of dockerfile

step (3) → docker build -t Myimage .
(-t ↓ tag)

→ docker ps -a

→ docker images

Step 4. Now, create container from the above image.

→ docker run -it --name Mycontainer myimg /bin/bash

→ cat /tmp/testfile

→ vi Dockerfile

```
FROM ubuntu
RUN echo "Technical guftgu" > /tmp/testfile


:wq
```

→ docker build -t Myimg .

→ docker images

→ docker run -it --name testcontainer Myimg /bin/bash

→ ls

→ cd tmp/

→ ls
testfile

→ cat testfile
Technical guftgu

LAB -3

→ vi Dockerfile

```
FROM ubuntu
WORKDIR /tmp
Run echo "Technical Guftgu" > /tmp/testfile
ENV myname bhupinderrajput
COPY testfile /tmp
ADD test.tar.gz /tmp

:wq
```

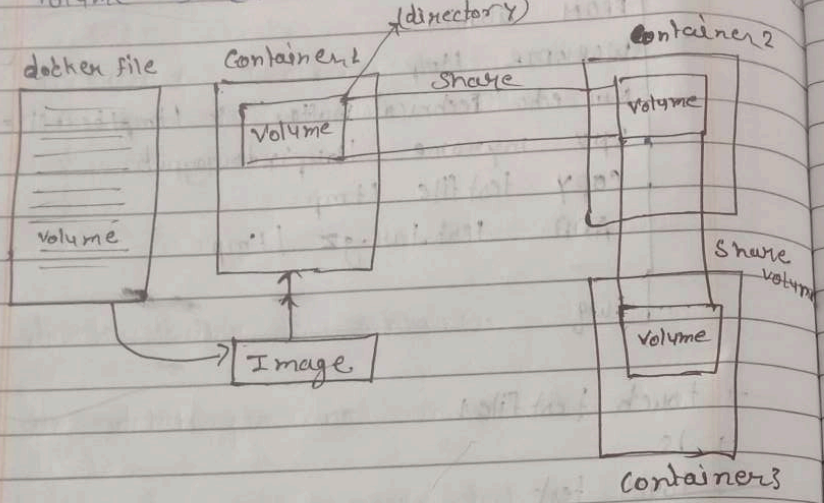
→ touch testfile1
→ ls
→ touch test
→ ls
→ tar -cvf test.tar test
→ ls
→ gzip test.tar
→ ls
→ rm -rf test
→ ls
→ docker build -t ~~newcon~~ newimage .
→ docker images
→ docker run -it --name newcontainer newimage /bin/bash
→ ls
→ cat testfile

```
→ echo $myname
```

Lecture - 28

Docker volume & How to share it

→ volume is basically a directory which act as a volume. Volume को हम container के साथ share कर सकते हैं।



→ हम जितने भी container के साथ Volume share करेंगे तो हम किसी भी Volume में कुछ change करे तो वह सभी ~~volume~~ container में दिखाई देगा या फिर कोई भी file किसी भी container में add करेंगे तो वह सभी में दिखाई देगा।

→ container delete करने पर भी Volume रहता है। या Volume का data बचा रहता है।

Important Rule Docker volume

Volume is simply a directory inside our container.

Firstly, we have to declare the directory as a volume and then share ~~it~~ volume.

Even if we stop container, still we can access volume.

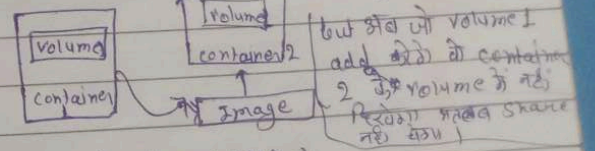
volume will be created in one container.

You can declare a directory as a volume only while creating container.

You can't create volume from existing container.

You can share one volume across any number of container.

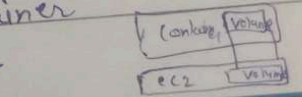
volume will not be included when you update an image.



You can mapped volume in two ways → (share or)

(1) container ↔ container

(2) Host ↔ container



Benefits of Volume

→ Decoupling container from storage
(it means container delete होने पे भी volume delete नहीं होता)

→ Share volume among different container

→ Attach volume to container

→ on deleting container volume does not delete.

LAB Container to Container

① create docker file and write.

→ vi Dockerfile

```
FROM ubuntu
VOLUME ["/myvolume1"]
:wq
```
(/myvolume1 → root directory)

Then create image from this dockerfile

→ `docker build -t myimage .`

Now create a container from this image & Run.

→ `docker run -it --name container1 myimage /bin/bash`

Now do ls, you can see myvolume1.

→ `ls`

Now share volume with another container

container1 ↔ container2

→ docker run -it --name container2 --privileged=true --volumes-from container1 ubuntu /bin/bash

Now after creating container2, myvolume1 is visible, whatever you do in one volume, can see from another volume,

→ touch /myvolume1/samplefile → container2 में जायेगी

→ docker start container1 → container1 में जाकर देखो कि samplefile वहाँ है या नहीं।

→ docker attach container1

→ ls /myvolume1

you can see samplefile here

→ exit

Commands

→ sudo su
→ yum update -y
→ yum install docker -y
→ service docker start
→ service docker status
→ vi Doc
→ touch file1 file2
→ ls
→ vi Dockefiler

```
FROM ubunty
VOLUME ["/myvolume"]
:wq
```

→ docker build -t myimage .
→ docker images
→ docker run -it --name container1 myimage /bin/bash
- ls
- cd myvolume/
- touch filex filey filez
- ls
- exit

→ docker run -it --name container2 privileged=true --volumes-from container1 ubunty /bin/bash

→ ls
• cd myvolume/
• ls
• exit
→ doc
→ docker start container1
→ docker attach container1
→ ls
→ cd myvolume/
→ ls

Now create volume by using command
or
without using Dockerfile

new command

→ docker run -it --name container3 -v /volume2 ubuntu /bin/bash

→ ls

→ cd /volume2 → touch file1 file2 file3

→ exit

∴ Now create one more container and share volume2

→ docker run -it --name container4 --privileged=true --volume-from container3 ubuntu /bin/bash

Now you are inside container4 do ls, you can see volume2

→ cd /volume2/ → ls

you can see file1, file2 and file3

→ touch samplefile1 samplefile2

→ ls

→ exit

Now start container3 and go inside the container

→ ~~service docker start~~

→ docker start container3

→ docker attach container3

→ ls

→ cd volume2/

→ ls

You can see the file samplefile1 and sample2 they creates in container4

Volumes share to Host to Container

or

Host to Container

→ verify file is /home/ec2-user
ls

→ docker run -it --name hostcont -v /home/ec2-user:/rajput --privileged=true ubuntu /bin/bash

→ cd /rajput
Do ls, now you can see all files of host machine.

→ touch rajputfile (in container)

exit

Now check in EC2 machine you can see this file.

Some other commands

→ docker volume ls {Show all volume}

→ docker volume create <volume name>
{to create a volume}

→ docker volume rm <volume name>

→ docker volume prune
{it removed all unused docker volume}

→ docker volume inspect <volume name>
[to check the information of volume]

→ docker container inspect <container Name>
{to check container Information}

Lec - 29 Docker Port Expose

Logical
0 - 65535 ports

How to access container using Internet

→ Login into AWS account create one Linux Instance.

✗ Now go to putty → Login as → EC2-user

→ sudo -s4

→ yum -y update

→ yum -y install docker

→ service docker start

→ docker run -td --name techserver -p 80:80 ubuntu

(host ← container; techserver; -p = port or publish)

→ docker ps (running container)

→ docker port techserver (container name)

↓ (show the open ports details)

o/p → 80/TCP → 0.0.0.0/80

→ docker exec -it techserver /bin/bash

(execute /bin/bash inside container but start new process)

→ apt-get update

→ apt-get install apache2 -y

→ cd /var/www/html

→ echo " Technical Cuffgy " > index.html

→ server apache2 start

Now copy the host machine public IP and paste any browser.

using jenkins

→ docker run -td --name myjenkins -p 8080:8080 jenkins

→ Note [allow 8080 port in security group]

Difference between docker attach and docker exec

→ Docker exec create a new process in the Container's environment while docker attach just connect the standard input/output of the main process inside the container to corresponding standard input/output error of current terminal.

→ docker exec is specifically for running new things in a already started container be it a shell or some other process.

what is the difference b/w expose and publish -p a docker?

Basically you have three options:-

(i) Neither specify [expose] nor [-p]

(ii) Only specify [expose]

(iii) specify [expose] and [-p]

(i) if you specify neither expose nor -p, the service in the container will only be accessible from inside the container itself.

(ii) if you expose a port the service in the container is not accessible from outside docker, but from inside other docker containers so this is good for inter-container communication.

(iii) if you expose and -p a port, the service in the container is accessible from anywhere, even outside docker

→ if you do -p but do not expose docker does an implicit expose this is because, if a port is open to the public it is automatically also open to the other docker containers Hence '-p' includes expose

LAB [Note - apt-get command ubuntu के लिए है]

(1.) docker run -td --name techserver -p 80:80 ubuntu

(2.) docker ps

(3.) docker port techserver
[to show the mapping port]

(4.) docker exec -it techserver /bin/bash
[to go the inside container]

(5.) > apt-get update [to update ubuntu]

(6.) > apt-get install apache2 -y

(7.) > cd /var/www/html

(8.) > echo "Technical Cyfigy" > index.html

(9.) > service apache2 restart
copy pubic Ip

(10.) > exit

(11.) docker run -td --name myjenkins -p 8080:8080 jenkins

(12.) §to add the port 8080 in Security group}

[then, copy public Ip address and paste browser.]